

Loss Pipeline: Mathematical Reference

Wave Equation PINN / KAN Experiment

Contents

1	Problem Statement	3
2	Material Models	3
3	Initial Condition Function	3
4	The Ansatz: Hard Initial Condition Enforcement	4
4.1	Motivation	4
4.2	Construction	4
4.3	Soft IC (use_ansatz = False)	4
5	PDE Residual	4
6	Causal PDE Loss	5
6.1	Motivation	5
6.2	Time-Slab Decomposition	5
6.3	Causal Weights	5
6.4	Causal Loss	6
7	Boundary Condition Loss	6
8	Initial Condition Loss (Soft)	6
9	Gradient-Norm Adaptive Weighting	6
9.1	Gradient Norms	7
9.2	Raw Weights	7
9.3	Exponential Moving Average	7
9.4	Ansatz mode	7
10	Full Loss Pipeline Summary	8

11 Causal Convergence Monitoring	8
11.1 In-training diagnostic (printed every <code>print_every</code> steps)	8
11.2 Saved outputs (per experiment run)	9
11.3 Interpretation	9
12 Optimizer Sequence	9
13 Systematic Parameter Sweep Experiments	10
13.1 Experimental Grid Dimensions	10
13.2 Detailed Architectural Sweep Configurations	11
13.3 Evaluation Metrics	11

1 Problem Statement

We solve the **1D elastic wave equation** on a non-dimensionalised space-time domain $\Omega = [x_{\min}, x_{\max}] \times [0, T]$:

$$\boxed{\rho(x) \frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial x} \left[E(x) \frac{\partial u}{\partial x} \right]} \quad (1)$$

where $u(x, t)$ is the displacement field, $\rho(x)$ is mass density, and $E(x)$ is Young's modulus (both normalised by their reference values $\rho_{\text{ref}}, E_{\text{ref}}$).

Initial conditions (ICs):

$$u(x, 0) = g(x) \quad (2)$$

$$\frac{\partial u}{\partial t}(x, 0) = 0 \quad (3)$$

Boundary conditions (BCs) — first-order absorbing (non-reflecting):

$$\frac{\partial u}{\partial t} - c(x) \frac{\partial u}{\partial x} = 0 \quad \text{at } x = x_{\min} \quad (4)$$

$$\frac{\partial u}{\partial t} + c(x) \frac{\partial u}{\partial x} = 0 \quad \text{at } x = x_{\max} \quad (5)$$

where $c(x) = V_p(x) = \sqrt{E(x)/\rho(x)}$ is the local P-wave velocity.

The network $\mathcal{N}_\theta(x, t)$ (PINN, FourierFeaturePINN, PirateNet, KAN, or WavKAN) approximates u via a modified output $\hat{u}$ described next.

2 Material Models

`wave/materials.py`

Three material configurations are used. All quantities are non-dimensionalised:

Model	$E(x)$	$\rho(x)$
Homogeneous	1	1
TwoLayer	$E_1(1 - \alpha) + E_2 \alpha$, $\alpha = \frac{1}{2}(1 + \tanh(x/w))$	1
MultiLayer	piecewise-linear via 6 tanh transitions	1

where $E_1 = 80/E_{\text{ref}}$, $E_2 = 120/E_{\text{ref}}$ for TwoLayer, and $w = 0.02/L_{\text{ref}}$ is the interface half-width.

3 Initial Condition Function

`gaussian_ic` in `wave/problem_data.py`

The prescribed initial displacement is a normalised first-derivative-of-Gaussian (a Ricker-like pulse):

$$g(x) = \frac{f'(x)}{\max_x |f'(x)| + \epsilon}, \quad f(x) = \exp\left(-\frac{(x - x_0)^2}{2\sigma_g^2}\right) \quad (6)$$

with $f'(x) = -\frac{x - x_0}{\sigma_g^2} f(x)$, $x_0 = 0$, $\sigma_g = 0.1$, $\epsilon = 10^{-12}$ (numerical stability).

The function is antisymmetric about x_0 , peaks near $\pm\sigma_g$, and is negligibly small outside $\approx 3\sigma_g$.

4 The Ansatz: Hard Initial Condition Enforcement

`apply_ansatz` in `wave/problem_data.py`

4.1 Motivation

Satisfying the IC with a soft loss term risks the network ignoring it early in training (when PDE gradients dominate). The *ansatz* bakes the IC directly into the network output.

4.2 Construction

Let $\mathcal{N}_\theta(x, t)$ be the raw network output. Define:

$$\hat{u}(x, t) = \underbrace{g(x) e^{-\frac{1}{2}(15t)^2}}_{\text{IC term (decays in time)}} + \underbrace{\tanh^2(25t)}_{\text{growth envelope}} \cdot \mathcal{N}_\theta(x, t) \quad (7)$$

Verification of IC satisfaction:

$$\hat{u}(x, 0) = g(x) \cdot 1 + 0 \cdot \mathcal{N}_\theta = g(x) \checkmark \quad (8)$$

$$\left. \frac{\partial \hat{u}}{\partial t} \right|_{t=0} = g(x) \cdot 0 + 2 \tanh(25t) \cdot \frac{25}{\cosh^2(25t)} \cdot \mathcal{N}_\theta \Big|_{t=0} = 0 \checkmark \quad (9)$$

The network $\mathcal{N}_\theta$ is completely free because the envelope zeroes it at $t = 0$.

4.3 Soft IC (`use_ansatz = False`)

When the ansatz is disabled:

$$\hat{u}(x, t) = \mathcal{N}_\theta(x, t) \quad (10)$$

and the ICs are enforced via penalty terms in the loss (Section 8).

5 PDE Residual

`compute_pde_residual` in `src/losses/wave_loss.py`

Given $\hat{u}$ from Section ??, the PDE residual at a collocation point (x_i, t_i) is:

$$R(x_i, t_i) = \rho(x_i) \hat{u}_{tt} - \frac{\partial}{\partial x} [E(x_i) \hat{u}_x] \quad (11)$$

All derivatives are computed exactly via `torch.autograd`:

1. Forward pass: $\hat{u} = \text{apply_ansatz}(\mathcal{N}_\theta(x, t), x, t)$

2. $\hat{u}_x, \hat{u}_t \leftarrow \nabla_{x,t} \hat{u}$ (first-order autograd, `create_graph=True`)
3. Stress: $\sigma = E(x) \hat{u}_x$
4. $\hat{u}_{tt} \leftarrow \partial_t \hat{u}_t$
5. $\sigma_x \leftarrow \partial_x \sigma$
6. $R = \rho(x) \hat{u}_{tt} - \sigma_x$

6 Causal PDE Loss

`causal_pde_loss` in `src/losses/wave_loss.py`

6.1 Motivation

The plain mean-squared PDE loss $\mathcal{L}_{\text{pde}} = \frac{1}{N} \sum_i R_i^2$ treats all time steps equally. But the wave equation is *causal*: the solution at large t depends on the solution at small t . Penalising late-time residuals before early-time is learned gives the optimizer contradictory gradients, helping $\hat{u} \equiv 0$ survive as a spurious minimum.

6.2 Time-Slab Decomposition

Partition $[0, T]$ into $M = 32$ equal slabs:

$$0 = \tau_0 < \tau_1 < \dots < \tau_M = T, \quad \tau_m = \frac{m}{M} T \quad (12)$$

Let the PDE loss in slab m be:

$$\mathcal{L}_m = \frac{1}{|\mathcal{S}_m|} \sum_{i: t_i \in [\tau_m, \tau_{m+1})} R(x_i, t_i)^2 \quad (13)$$

6.3 Causal Weights

Define weights that depend on how well all earlier slabs are solved:

$$w_m = \exp\left(-\varepsilon \sum_{k=0}^{m-1} \mathcal{L}_k\right) \quad (14)$$

with $\varepsilon = 0.5$ (the `tolerance` parameter) and $w_0 = 1$ by convention (empty sum = 0).

Properties:

- $w_0 = 1$ always — the first slab (near IC) is always fully weighted.
- If $\mathcal{L}_{m-1}$ is large (slab $m-1$ unsolved), $w_m \approx 0$ — slab m contributes almost no gradient.
- As training progresses and $\mathcal{L}_k \rightarrow 0$ for $k < m$, $w_m \rightarrow 1$ — the frontier propagates forward in time.

The weights are computed from `.detach()`ed losses, so they are treated as constants in backprop (no gradient flows through w_m).

6.4 Causal Loss

$$\mathcal{L}_{\text{pde}} = \frac{1}{M} \sum_{m=0}^{M-1} w_m \mathcal{L}_m \quad (15)$$

7 Boundary Condition Loss

`losses_gradnorm` in `src/losses/wave_loss.py`

The absorbing BCs (4)–(5) are enforced via a squared residual over N_{bc} boundary collocation points with random $t \in [0, T]$:

$$\mathcal{L}_{\text{bc}} = \frac{1}{N_{\text{bc}}} \sum_i \left[\underbrace{(\hat{u}_t - c \hat{u}_x)^2}_{\text{left ABC}} \Big|_{x=x_{\min}} + \underbrace{(\hat{u}_t + c \hat{u}_x)^2}_{\text{right ABC}} \Big|_{x=x_{\max}} \right]_{t_i} \quad (16)$$

The sign flip between left and right encodes the direction of wave travel: leftward waves exit at $x_{\min}$, rightward waves exit at $x_{\max}$.

8 Initial Condition Loss (Soft)

`ic_loss` in `src/losses/wave_loss.py`

Used only when `use_ansatz = False`. Penalises violations of both ICs (2)–(3) at N_{ic} random x points with $t = 0$:

$$\mathcal{L}_{\text{ic}}^{\text{disp}} = \frac{1}{N_{\text{ic}}} \sum_i [\hat{u}(x_i, 0) - g(x_i)]^2 \quad (17)$$

$$\mathcal{L}_{\text{ic}}^{\text{vel}} = \frac{1}{N_{\text{ic}}} \sum_i \left[\frac{\partial \hat{u}}{\partial t}(x_i, 0) \right]^2 \quad (18)$$

$$\mathcal{L}_{\text{ic}} = \mathcal{L}_{\text{ic}}^{\text{disp}} + \mathcal{L}_{\text{ic}}^{\text{vel}} \quad (19)$$

$\hat{u}_t$ at $t = 0$ is computed via autograd with `requires_grad=True` on the t -input tensor.

IC pre-scaling. Before returning from `losses_gradnorm`, the IC loss is multiplied by a fixed constant $s = 10$:

$$\tilde{\mathcal{L}}_{\text{ic}} = s \mathcal{L}_{\text{ic}}, \quad s = 10 \quad (20)$$

This ensures GradNorm (Section 9) sees a non-negligible IC gradient from the first iteration, preventing the trivial solution $\hat{u} \equiv 0$ from being entrenched before the IC loss is weighted up.

9 Gradient-Norm Adaptive Weighting

`GradNormWeighter.compute_weights` in `src/losses/wave_loss.py`

Following Wang et al. (2023), the total loss is a weighted sum:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}} + \lambda_{\text{ic}} \tilde{\mathcal{L}}_{\text{ic}} \quad (21)$$

where the weights λ are updated every $\Delta = 100$ Adam steps.

9.1 Gradient Norms

For each loss term $\mathcal{L}_k \in \{\mathcal{L}_{\text{pde}}, \mathcal{L}_{\text{bc}}, \tilde{\mathcal{L}}_{\text{ic}}\}$, compute the ℓ^2 norm of gradients with respect to all network parameters θ :

$$\|\nabla_{\theta} \mathcal{L}_k\|_2 = \left(\sum_j \left(\frac{\partial \mathcal{L}_k}{\partial \theta_j} \right)^2 \right)^{1/2} \quad (22)$$

9.2 Raw Weights

The raw (un-smoothed) weight for term k is:

$$\hat{\lambda}_k = \frac{\sum_{k'} \|\nabla_{\theta} \mathcal{L}_{k'}\|_2}{\|\nabla_{\theta} \mathcal{L}_k\|_2} \quad (23)$$

Intuitively: a loss term whose gradients are small gets a larger weight so that it contributes comparably to the total gradient signal.

9.3 Exponential Moving Average

To avoid rapid oscillations, weights are smoothed:

$$\lambda_k^{(n)} = \begin{cases} \hat{\lambda}_k^{(n)} & n = 0 \\ \alpha \lambda_k^{(n-1)} + (1 - \alpha) \hat{\lambda}_k^{(n)} & n > 0 \end{cases} \quad (24)$$

with $\alpha = 0.9$. Between update steps ($n \not\equiv 0 \pmod{\Delta}$), the previous λ_k is reused unchanged.

9.4 Ansatz mode

When `use_ansatz = True`, the IC is satisfied by construction, so:

$$\lambda_{\text{ic}} = 0, \quad \hat{\lambda}_k = \frac{\|\nabla_{\theta} \mathcal{L}_{\text{pde}}\|_2 + \|\nabla_{\theta} \mathcal{L}_{\text{bc}}\|_2}{\|\nabla_{\theta} \mathcal{L}_k\|_2}, \quad k \in \{\text{pde}, \text{bc}\} \quad (25)$$

10 Full Loss Pipeline Summary

Forward:

$$\hat{u}(x, t) = g(x) e^{-\frac{1}{2}(15t)^2} + \tanh^2(25t) \cdot \mathcal{N}_\theta(x, t) \quad [\text{or just } \mathcal{N}_\theta \text{ if no ansatz}]$$

PDE residual at each (x_i, t_i) :

$$R_i = \rho(x_i) \hat{u}_{tt}(x_i, t_i) - \partial_x [E(x_i) \hat{u}_x(x_i, t_i)]$$

Causal slab loss $m = 0, \dots, M-1$:

$$\mathcal{L}_m = \text{mean}_{i \in \text{slab}_m} R_i^2$$

$$w_m = \exp\left(-\varepsilon \sum_{k < m} \mathcal{L}_k^{\text{det}}\right)$$

$$\mathcal{L}_{\text{pde}} = \frac{1}{M} \sum_m w_m \mathcal{L}_m$$

BC loss:

$$\mathcal{L}_{\text{bc}} = \text{mean}[(\hat{u}_t - c \hat{u}_x)^2|_{x_{\min}} + (\hat{u}_t + c \hat{u}_x)^2|_{x_{\max}}]$$

IC loss (soft mode only):

$$\mathcal{L}_{\text{ic}} = \text{mean}[(\hat{u}(x, 0) - g(x))^2] + \text{mean}[(\hat{u}_t(x, 0))^2], \quad \tilde{\mathcal{L}}_{\text{ic}} = 10 \mathcal{L}_{\text{ic}}$$

GradNorm weights(*every100steps*) :

$\|\nabla_\theta \mathcal{L}_k\|_2$ computed for each term

$$\hat{\lambda}_k = \frac{\sum_{k'} \|\nabla_\theta \mathcal{L}_{k'}\|_2}{\|\nabla_\theta \mathcal{L}_k\|_2}, \quad \lambda_k \leftarrow 0.9 \lambda_k + 0.1 \hat{\lambda}_k$$

Total loss (Adam step):

$$\mathcal{L}_{\text{total}} = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}} + \lambda_{\text{ic}} \tilde{\mathcal{L}}_{\text{ic}}$$

After Adam: L-BFGS fine-tuning

Fixed λ_k (inherited from end of Adam phase)

11 Causal Convergence Monitoring

`get_causal_frontier` · `src/losses/wave_loss.py`, `src/train.py`, `wave/run_experiment.py`

11.1 In-training diagnostic (printed every `print_every` steps)

After each forward pass through `causal_pde_loss`, the weight vector $\mathbf{w} = (w_0, w_1, \dots, w_{M-1})$ is stored in the module-level `_causal_state` dict.

`get_causal_frontier` reads this vector and reports two quantities:

1. **Frontier count** (coarse indicator):

$$F(\delta) = \#\{m : w_m \geq \delta\}, \quad \delta = 0.1 \tag{26}$$

A chunk is “reached” when its causal weight exceeds 10%. The bar in the log shows F/M visually.

2. $w_{\min}$ (paper’s convergence criterion):

$$w_{\min} = \min_m w_m \quad (27)$$

When $w_{\min} \geq 0.99$, the output changes to **CONVERGED**, meaning:

$$\sum_{k=0}^{M-2} \mathcal{L}_k \approx 0 \quad \Longleftrightarrow \quad \text{PDE residual is near-zero in *all* time slabs} \quad (28)$$

11.2 Saved outputs (per experiment run)

`train_adam` tracks causal data at every iteration and saves to disk:

File	Contents	Frequency
<code>causal_convergence.json</code>	<code>converged</code> , <code>final_w_min</code> , <code>w_min_history</code> , <code>chunk_snapshots</code>	end of Adam phase
<code>causal_wmin.png</code>	$w_{\min}$ vs. Adam iteration	plot
<code>causal_heatmap.png</code>	w_m per chunk vs. iteration (heatmap)	plot

`causal_wmin.png` plots $w_{\min}(n)$ over Adam steps. The paper’s convergence criterion is $w_{\min} \rightarrow 1$; a horizontal dashed line marks the threshold 0.99.

`causal_heatmap.png` shows w_m as a 2D image with Adam iteration on the x -axis and chunk index m on the y -axis. Colour encodes weight (yellow ≈ 1 , purple ≈ 0). Healthy causal training produces a “yellow wavefront” that sweeps upward (from chunk 0 to chunk $M - 1$) as training progresses.

11.3 Interpretation

- $w_{\min}$ stays near 0 throughout $\Rightarrow$ 15,000 Adam iterations is not enough; increase `adam_iterations` or decrease ε .
- $w_{\min}$ rises then plateaus below 0.99 $\Rightarrow$ partial convergence; L-BFGS may push it over the threshold.
- $w_{\min} \geq 0.99$ at end of training $\Rightarrow$ full causal convergence achieved; paper criterion satisfied.
- Heatmap has a sharp diagonal boundary $\Rightarrow$ clean causal propagation (ideal).
- Heatmap has a fuzzy or non-monotone boundary $\Rightarrow$ consider increasing ε or `n_chunks`.

12 Optimizer Sequence

`src/train.py`

1. **Adam phase** (15,000 iterations, `lr = 0.001`):

- GradNorm weights updated every 100 steps.
- Best checkpoint saved whenever total loss improves.
- Final weights λ_k^* passed to L-BFGS.

2. **L-BFGS phase** (2,000 iterations, Strong Wolfe line search):
 - Weights λ_k fixed at λ_k^* from Adam.
 - Same loss function; provides second-order curvature information.
 - Best checkpoint saved independently.
3. **L-BFGS only** (5,000 iterations, alternative mode):
 - Adam phase skipped entirely; weights initialised to $(1, 1, 1)$.

13 Systematic Parameter Sweep Experiments

`wave/run_experiment.py`

To compare model capacities, optimization trajectories, and training behaviors under varying conditions, a multi-dimensional parameter sweep is executed. The total grid consists of 360 unique experiment runs.

13.1 Experimental Grid Dimensions

The sweep parameters are categorized along four main dimensions:

1. **Material Models (3 profiles)**:
 - **Homogeneous**: Constant wave speed ($E(x) = 1, \rho(x) = 1$).
 - **Two-Layer**: Single interface with a smooth density/modulus transition.
 - **Multi-Layer**: Heavily stratified medium with 6 smooth transitions.
2. **Model Architectures (5 architectures, 30 configurations)**:
 - **PINN (4 configurations)**: Vanilla Multi-Layer Perceptrons.
 - **FourierFeaturePINN (8 configurations)**: Random Fourier mapping combined with an MLP.
 - **PirateNet (6 configurations)**: Custom residual skips with Fourier features.
 - **KAN (4 configurations)**: Kolmogorov-Arnold Networks built with B-splines.
 - **WavKAN (8 configurations)**: Kolmogorov-Arnold Networks built with wavelets.
3. **Ansatz Initial Condition Enforcement (2 modes)**:
 - **Hard Enforcement** (`use_ansatz = True`): Satisfies IC by construction.
 - **Soft Enforcement** (`use_ansatz = False`): Enforced via GradNorm-penalized boundary loss terms.
4. **Optimization Strategy (2 sequences)**:
 - **Adam + L-BFGS**: 15,000 steps of Adam with dynamic GradNorm scaling followed by 2,000 steps of L-BFGS.
 - **L-BFGS Only**: 5,000 steps of L-BFGS directly from random initialization.

The full combinatorial sweep corresponds to:

$$\text{Total Runs} = 3 \text{ Materials} \times 30 \text{ Configs} \times 2 \text{ Ansatz Modes} \times 2 \text{ Optimizers} = 360 \text{ Runs} \quad (29)$$

13.2 Detailed Architectural Sweep Configurations

The 30 specific configurations are defined as follows:

Architecture	Slabs/Layers	Width	Fourier Features (N_f)	Hyperparameters / S
PINN	2, 3, 4, 5	32, 64, 128, 128	–	–
FourierFeaturePINN	2, 3, 4, 5	32, 64, 128, 128	64, 128, 128, 256	$\sigma \in \{3.0, 7.0\}$
PirateNet	2, 3, 4	32, 64, 128	64, 128, 128	$\sigma \in \{3.0, 7.0\}$
KAN	1, 2, 3, 4	16	–	Grid $G \in \{3, 5\}$, Spline
WavKAN	1, 2, 3, 4	16	–	Wavelet $\in \{\text{Morlet, Mex}\}$

13.3 Evaluation Metrics

For each run, the metrics evaluated are:

1. **Relative $\mathcal{L}^2$ Error (%)**: Evaluated against a reference solution u_{ref} generated via an explicit finite-difference solver:

$$\text{Rel } \mathcal{L}^2 \text{ Error } (\%) = \frac{\|u_{\text{pred}} - u_{\text{ref}}\|_2}{\|u_{\text{ref}}\|_2} \times 100 \quad (30)$$

This is computed independently for the Adam checkpoint and the final L-BFGS checkpoint.

2. **Causal Convergence**: Stored in `causal_convergence.json`, verifying whether the minimum causal weight reached the convergence threshold ($w_{\text{min}} \geq 0.99$).